

05 GREEDY

Lab 4: Greedy Best-First Search

Travel from S to G.

Road Costs: $S \rightarrow A = 2$

$S \rightarrow B = 1$

$A \rightarrow G = 10$

$B \rightarrow G = 4$

You are also given a heuristic h - an estimate of the remaining distance, like straight-line distance on a map.

$h(S) = 4$; $h(B) = 3$

$h(A) = 2$; $h(G) = 0$

Greedy expands whichever node has the smallest h , ignoring what has already been spent.

Answer this: The two routes cost 12 and 5. Greedy returns 12. The heuristic never overestimates, so it is not the heuristic's fault, which exact line of code is responsible?

⇒

Python Code

```
import heapq
```

```
graph = {
```

```
    'S': [( 'A', 2 ), ( 'B', 1 )],
```

```
    'A': [( 'C', 10 )],
```

```
    'B': [( 'C', 4 )],
```

```
    'C': []
```

```
}
```

```
h = S
```

```
    'S': 4,
```

```
    'A': 2,
```

```
    'B': 3,
```

```
    'C': 0
```

```
}
```

```
frontier = [(h['S'], 'S', [ 'S' ], 0)]
```

```
while frontier:
```

```
    node, path, cost = heapq.heappop(frontier)
```

```
    if node == 'C':
```

```
        print ("Path:", " → ".join(path))
```

```
        print ("Cost:", cost)
```

```
        break
```

```
    for neighbor, road_cost in graph[node]:
```

```
        new_cost = cost + road_cost
```


heapq.heappush (

frontier,

(h [neighbour], neighbour, path + [neighbour]

new_cost) . [(1, 'B'), (2, 'A')] = 2

)

Output

Path : $S \rightarrow A \rightarrow C$

Cost : 12

Answer

The responsible line is :

heapq.heappush (frontier,

(h [neighbour], neighbour,

path + [neighbour], new_cost)

Especially :

h [neighbour]

is placed 1st in the priority queue

So, priority queue choose the node with smallest heuristic. Instead of choosing the node with the smallest total cost.

So, $h(A) = 2$

$h(B) = 3$

Greedy chooser: A

and gets $S \rightarrow A \rightarrow C = 12$

Instead of : $S \rightarrow B \rightarrow C = 5 + 8 = 13$